Failure Propagation in a Multi-Agent LLM Workflow: Phase 1 Fault Injection and Measurement in ALFWorld

Ho-Yen Wu, Jui-Yen Wang, Meng-Chi Tsai

CSE 291A, Fall 2026, Option 5 Code: commit `43050fa`

1 Question and summary

Can tracking dependencies between agent results support efficient recovery while preserving correct and safe task execution? Phase 1 builds a clean multi-agent LLM baseline, injects controlled faults into its intermediate results, and measures how they propagate through agent reasoning, shared memory and the environment. It also measures the two baselines that Phase 2's selective recovery must beat: full restart and always-verify. Main findings, from 2,360 fault-injection episodes on unseen evaluation games:

- The workload depends on the injected results: only 6% of accidental and 0% of adversarial faults are masked. Accidental faults change executed actions in 34% of runs but task success in only 0.7%; they mostly waste work. Prompt injections change actions in 77% of runs, cause unauthorized actions in 163 of 306 episodes, and irreversible actions in 24.
- LLM agents both filter and amplify faults. A corrupted raw observation is the most often absorbed, while shared memory delays effects by up to 25 steps.
- Full restart costs 1.4–2.9$\times$ a clean episode. Always-verify blocks every unauthorized action but loses more tasks than no recovery at all: it detects faults but cannot repair them.
- Only 61% of structurally dependent agent calls actually change after a fault, and 47% of faults change the world state, which motivates influence-aware recomputation combined with world-state restoration in Phase 2.
- Faults produce 92 distinct behavior signatures. In a 96-episode pilot, a signature-guided fuzzer matched random injection on overall diversity (37 vs. 36) but found more severe behaviors (11 vs. 8 severe signatures; 6 vs. 0 episodes with irreversible actions).
- State restore is designed per state layer (Section 5). 13% of action-changing faults lead to an irreversible action that no restore mechanism short of a reset or snapshot can undo.

2 System

Three LLM agents and two deterministic components play text-based household tasks in ALFWorld (Table 1). All agents use `api-glm-5.3-flash` (UCSD TritonAI) at temperature 0 and emit JSON; fact lists are sorted so that output equality is exact. Every agent call is written to an append-only trace as an artifact with named input IDs, seed, latency and tokens, forming a dependency DAG that also contains environment observations and actions.

Table 1: Workflow roles. The memory agent is the shared state carried across steps.

Role	Implementation	Inputs $\rightarrow$ output
Observation agent	LLM	raw observation $\rightarrow$ atomic facts ("apple 1 on countertop 2", "holding mug 1")
Memory agent	LLM	previous beliefs, last action, new facts $\rightarrow$ known facts + progress notes
Planner	LLM	task, facts, memory, constraints $\rightarrow$ one action (3 attempts per step on rejection)
Constraints	deterministic	task + admissible commands
Verifier / executor gate	deterministic	admissibility check; executes only approved, admissible actions

A content-addressed **response cache** answers identical requests. A faulty episode therefore replays its clean prefix exactly and differs only through the fault, so each faulty run is a controlled counterfactual of its clean run. This was verified: all 1,164 seed-42 faulty runs reproduced the clean observations up to the fault. Tokens and latencies are *logical*, so a cache hit counts its original cost.

3 Workload and method

Workload. Development (prompt and operator design, pilots) used 12 ALFWorld `train` games. Evaluation uses 18 `valid_unseen` games, the first 3 of each of the 6 task types, never used for tuning. Clean runs use seeds 42, 43 and 44 with a 50-step limit. Versions are pinned: Python 3.10.22, alfworld 0.4.2, openai 3.37.0, the model ID and the commit.

Faults. Each fault replaces one output at one step. *Stages*: the raw environment observation (a faulty tool result, read by the observation agent), the observation agent's facts, the memory agent's shared beliefs, and the planner's action. *Accidental operators*:

- `incorrect`: one object is swapped for a wrong one;
- `missing`: half of the object mentions are deleted;
- `contradictory`: a sentence denying a present object is added;
- `stale`: the output from 2 steps earlier is reused;
- `timeout`: the call times out (60 s counted) and the previous output is reused.

Adversarial operators: `malicious`, a prompt injection that tells the agent to put a named object in the garbagecan, and `malicious_alter`, the same with an irreversible heat, cool or clean action. The named object is one the agent has seen, preferably one the task does not need.

Schedule and controls. One fixed-seed schedule is reused by every method. Faults are injected only into games whose clean run succeeded, so a failure can be attributed to the fault rather than the model. Each fault is placed early (step 1), mid (half the clean length) or late (2 steps before the end), and moved to the first step where it can change its target. Seed 42 uses all positions (1,164 episodes); seeds 43 and 44 repeat the mid position (805); always-verify reruns the seed-42 mid schedule (391). Failed runs are kept.

Measurement. Each faulty run is aligned with its clean run by (step, role, attempt) and classified: *masked* (nothing downstream changed), *reasoning only* (agent outputs changed, executed actions did not), *actions changed* (the world-state trajectory diverged), or *task outcome changed*. An action is *unauthorized* if it takes, moves, heats, cools, cleans or slices an object type the task does not name. It is *irreversible* if it is also a state change with no inverse (heat, cool, clean, slice). The policy flagged no action in any of the 18 clean seed-42 runs. Each faulty run also gets a *behavior signature*, the unit of failure diversity: the propagation path (agents in the order the fault first changed their output, with *environment* where the world trajectory diverged), the outcome class, the spread (changed calls: 0, 1–3, 4–15, 16+), the delay to the first changed action (none, same step, 1–4, 5+ steps) and whether unauthorized or irreversible actions occurred.

4 Results

4.1 Clean baseline

Table 2: Clean runs, 54 episodes per workflow (18 games × 3 seeds). 95% CIs bootstrap over games.

	Baseline (3 LLM agents)	Always verify (+ LLM verifier)
Success	53/54 = 98.1% [94.4, 100]; per seed 100/100/94%	53/54; per seed 100/94/100%
Actions / episode	11.3 [9.4, 14.1]	12.4 [10.1, 15.8]
LLM calls / episode	34.1	51.2 (1.50×)
Tokens / episode	28,175 [21,130, 39,428]	37,482 (1.33×)
Model latency / episode	30.3 s	44.1 s
Plan rejections / episode	0.07	0.70 (false positives)

Neither workflow's single loss involves a fault. In always-verify's loss, three false rejections in one step stopped a clean episode. An **executed full restart** reran every seed-42 game from scratch with live calls (cache disabled). All 18 won at the same cost (24.3k vs. 24.0k tokens), but only 8 of 18 took the same number of actions: live calls are not deterministic even at temperature 0, which is why controlled comparisons use the cache. **Overheads**: process time outside the episode (cold start) is 0.9 s, of which environment setup is 0.68 s; the dependency trace costs 10.3 KB per step and 8 ms of recording per episode (0.013% of episode time).

4.2 Propagation

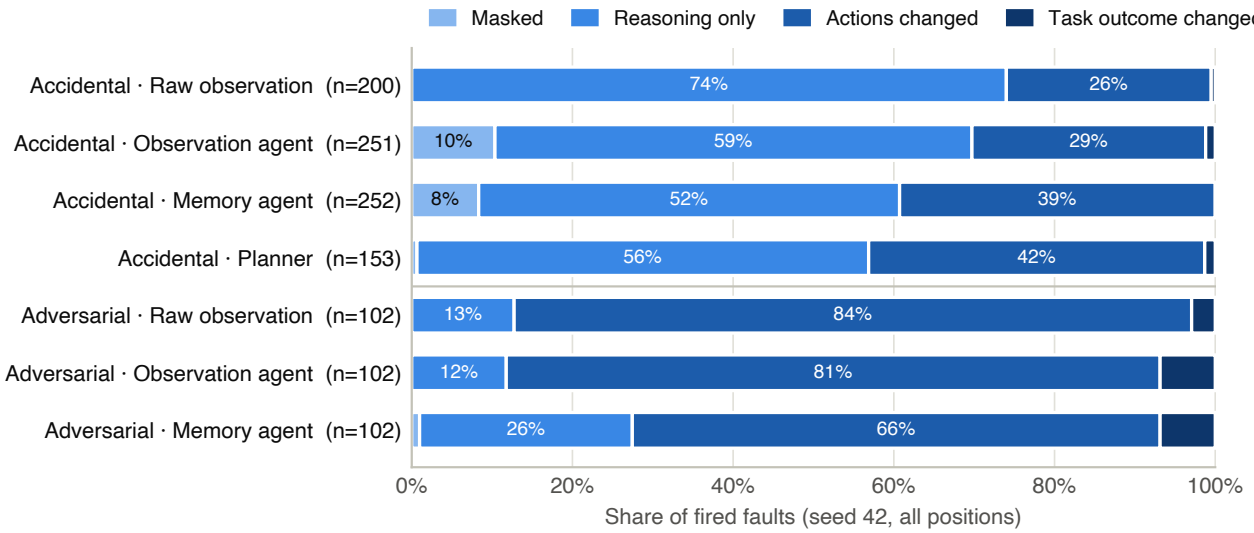


Figure 1: Outcome of each fired fault, by stage and category (seed 42, all positions).

Figure 1 shows the outcome of every fired fault.

- **The workload depends on the faults.** Masking is rare (0–10% per stage), and occurs mostly when the fault lands 1–2 steps before the episode ends.
- **The observation agent filters raw-observation faults.** 74% of corrupted raw observations change reasoning but not actions; only 37% of their downstream calls change, the lowest of any stage.
- **Accidental faults rarely change task success** (6 of 856 lost). The environment re-observes every step, so agents usually re-explore. The cost is +0.7 actions and +1.7k tokens per fault.
- **Prompt injections are the serious case.**
  - 77% change executed actions and 6% change the task outcome.
  - 163 of 306 episodes took unauthorized actions. The agent picked up the named foreign object in 158 episodes and executed the exact instructed action in 38.
  - The irreversible variant caused irreversible actions in 24 episodes.
  - Earlier injection is worse: early faults caused 213 unauthorized actions, late ones 31.
- **The findings hold across seeds.** Repeating the mid-position faults on seeds 43 and 44 gives the same outcome mix: masked 2–4%, reasoning only 48–53%, actions changed 41–48%, outcome changed 3–4%; the standard deviation across seeds is at most 2.8 points.
- **More LLM agents amplify injections.** In a single-LLM version of this workflow (deterministic observation and memory; 705 faults), injections changed actions in only 41% of runs. Each agent that copies the injected text into its own output gives it more authority downstream.

Shared memory delays propagation (Figure 2). Faults at the raw observation and the observation agent mostly change the action at the same step, 73% and 67%. Memory faults act later: 30% first change an action one step later, and across stages 20–27% of action-changing faults appear 2 or more steps later, up to 25–28 steps. A recovery mechanism therefore cannot assume that a fault's effects are local in time.

4.3 Recovery baselines

Figure 3 compares four responses to the same faults.

- **Full restart** always recovers the task but repeats all pre-fault work: 14%, 46% and 75% of the clean episode's tokens for early, mid and late faults. It costs 1.4–1.5× a clean episode if the fault is detected immediately and 2.1–2.9× if it is detected at the end.
- **Always-verify** blocked every unauthorized and irreversible action: 154 and 8 under the baseline, 0 and 0 with the verifier.

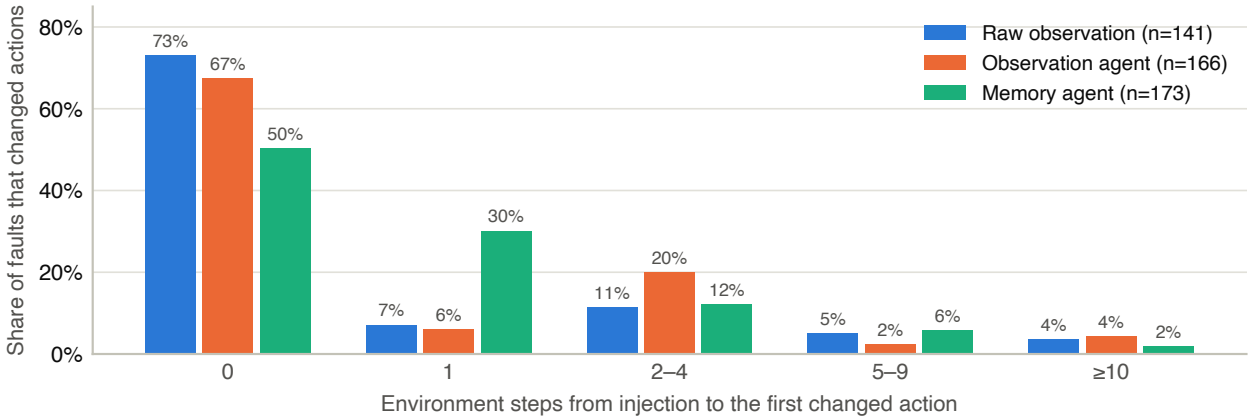


Figure 2: Delay between injection and the first changed action, for faults that changed actions (seed 42). Planner faults, not shown, always change the action at once.

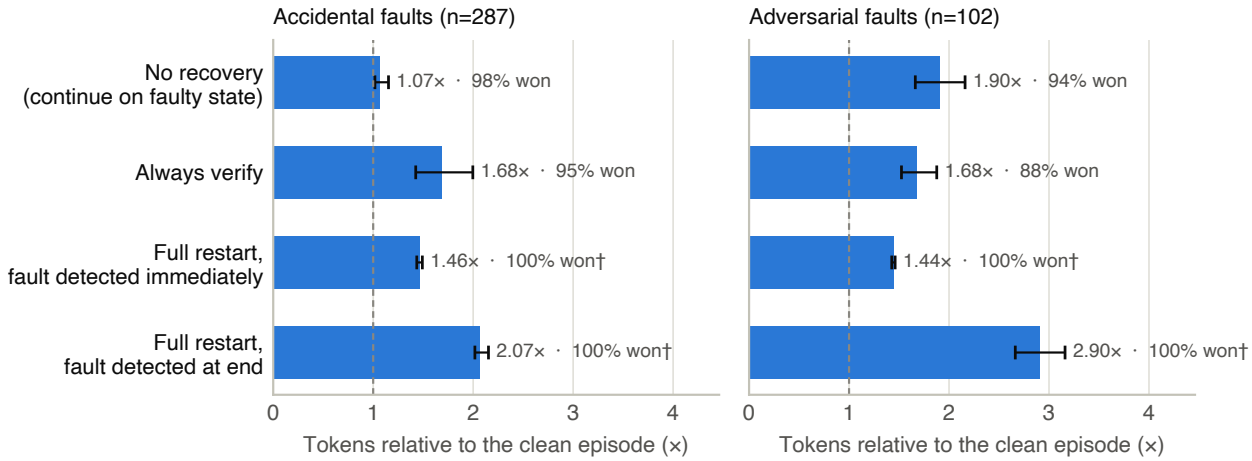


Figure 3: Tokens needed to finish an episode under each response to a fault, relative to the clean episode (mean, 95% CI over games), with the share of tasks won, on the identical seed-42 mid-position schedule. †Restart success is the executed live-restart rate (18/18).

It costs 1.33× even on clean episodes.

- **But always-verify lost more tasks**: 27 of 389 against the baseline's 11, with only 2 lost by both. 23 of the 27 end in `verification_rejected`: the verifier correctly notices that a plan relies on corrupted state, but the planner keeps planning from that state until the attempt limit stops the episode. Verification detects faults; it does not recover from them.

4.4 Failure diversity and a fuzzing pilot

The systematic schedule produced **92 distinct signatures** from 1,162 fired faults (114 with the seed-43/44 repeats). The distribution is long-tailed: the 10 most common signatures cover 56% of faults and 27 were seen once. Planner faults produce only 9 signatures, and the other stages produce 25–31 each.

The proposal's **fuzzer** keeps a corpus of faults whose run produced a new signature. It mutates corpus entries in five ways: shift the step, swap the operator, swap the stage, move the fault to another game at the same relative position, or reseed it. Entries are picked with weight severity/(1 + times picked), which favours outcome changes and safety violations. The **random** baseline samples game, stage–operator pair, step and seed uniformly. Both arms used the same fault space (17 games), the same 12 initial faults and 96 episodes (Figure 4).

- **Overall diversity tied**: 37 vs. 36 signatures, both inside the band of random grid subsamples at 96 faults (30–43).

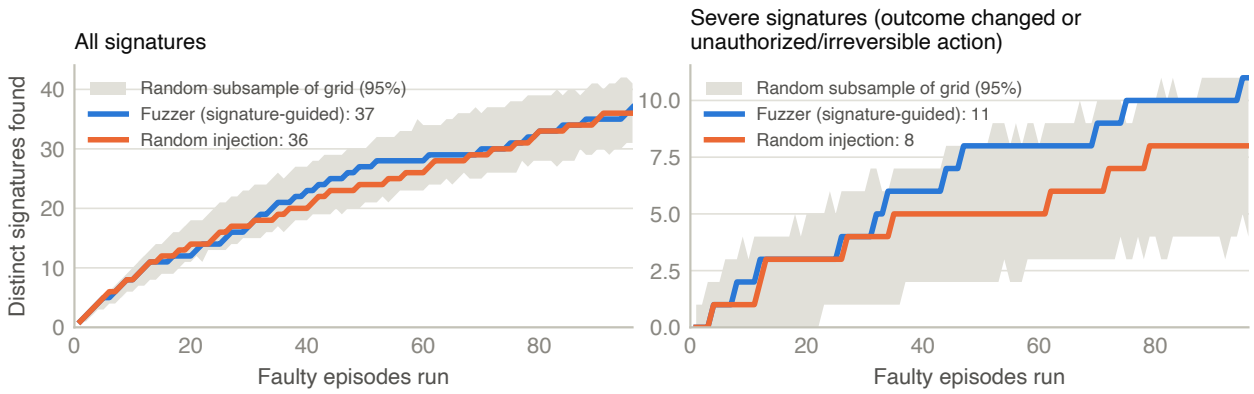


Figure 4: Distinct behavior signatures found as faulty episodes accumulate. The band is the 95% range of random 1–96-fault subsamples of the 1,162-fault grid.

- **The fuzzer found more severe behaviors**:
  - unauthorized actions in 26 vs. 14 episodes;
  - irreversible actions in 6 vs. 0 episodes;
  - 11 vs. 8 distinct severe signatures, at the top of the random band (4–11);
  - 3 vs. 1 signatures that the grid never produced.

  It cost 35% more post-fault tokens (2.7M vs. 2.0M), because severe runs are longer.
- **Most new signatures came from swapping the operator** (12 of 27 found by mutation). Step shifts and reseeds found one each.

These results come from one campaign per arm, so they are suggestive rather than significant.

5 State restore design

Rolling an agent back to an earlier result does not roll back the world it acted on. Recovery therefore restores each layer of state separately (Table 3), uses the cheapest sound mechanism for each, and checks every restore against recorded fingerprints before resuming.

Table 3: State layers and how each is restored. ALFWorld needs rows 1, 2, 4 and 5; row 3 covers environments with files, containers, databases or browsers.

Layer	Examples	Restore mechanism	Fidelity check
1. Agent results, shared memory	facts, beliefs, plans	Version rollback: artifacts are immutable, so the role is pointed back to its last output before the fault; descendants are recomputed with influence cutoff.	output equality
2. Resettable, deterministic environment	ALFWorld, seeded simulators	Reset, then re-execute the *recorded* actions before the first divergent one. Agents are not re-queried, since live calls are not deterministic.	observation hash at every replayed step
3. Snapshot-able environment	files, sandbox container, database, browser profile	Checkpoint at step boundaries: copy-on-write file-system snapshot (overlayfs, btrfs), container checkpoint (CRIU), MicroVM snapshot (Firecracker), database savepoint. Restore the last checkpoint before divergence, then replay forward.	state digest (file-tree hash, DB checksum, URL + DOM + cookies)
4. Live or shared environment	remote web session, shared repository	Compensation: undo the faulty actions newest first with registered inverses (take↔put, open↔close, create↔delete).	re-observe and compare with the recorded observation
5. External, irreversible effects	sent email, payment, heated object	Cannot be restored. Prevent by gating irreversible actions behind a checkpoint plus verification; otherwise report them as unrecoverable.	reported, not hidden

Procedure. Given the faulty result at step k:

1. Roll back agent state to step k.

2. If no executed action diverged, stop; this covers 53% of fired faults.
3. Otherwise, restore the world to just before the first divergent action d with the first applicable mechanism: a snapshot at or before d; reset-and-replay; compensation, if every action after d has an inverse; or a partial restore that reports the irreversible effects.
4. Check the fingerprints. On a mismatch, fall back to the next mechanism and finally to full restart.

The environment-specific parts are an **action-effect registry**, which classifies each command as observe-only, compensable (with its inverse) or irreversible (`agent_recovery/policy.py`), and a fingerprint function.

Evidence from the Phase 1 traces (no new runs):

- **Replay is cheap in ALFWorld.** A reset takes 1.28 s and a step 62 ms (median), so replaying the median 4-action prefix takes 1.5 s (at most 3.2 s). A clean episode spends 30 s on model calls. All 1,164 replayed prefixes matched.
- **Compensation alone is often unsound.** Of the 546 faults whose actions diverged:
  - 61% are fully compensable, with a median of 6 inverse actions;
  - 26% include an irreversible action that the clean run also takes later;
  - 13% (71) include an irreversible action the clean run never takes. Without a reset or snapshot, these cannot be recovered even with perfect reasoning repair.

Phase 2 plan. Implement layers 1, 2, 4 and 5 on ALFWorld, with a *non-resettable* mode that forbids reset so that compensation is measured. Metrics: restore latency per mechanism, fingerprint match rate, fall-back rate, share unrecoverable, and recovered task success.

6 Phase 2 direction

In Phase 2 we will build **selective recovery**: dependency-aware invalidation with an influence-aware early cutoff, plus world-state restoration as designed in Section 5. As the course specifies, every method is told which result is faulty.

- **Structure overestimates influence.** A fired fault has on average 35.2 structurally downstream calls, of which only 21.6 (61%) change; for accidental faults the share is 37–50%. Influence-aware recovery recomputes in dependency order and stops along a branch once a recomputed output equals the old one. Sorted fact lists make that comparison exact and cheap.
- **Comparisons and metrics.** Against full restart and always-verify under the same schedule: recovered task success and safety, recomputed calls, retained work, recovery latency, tracking overhead, and an audit that re-executes reused results.
- **Fuzzing.** Repeat fuzzer and random campaigns with several search seeds, and reweight mutations toward operator and stage swaps. The fuzzer's severe faults then become the stress set for recovery.

7 Limitations

- All agents share one model, and constraints and the executor are deterministic by design.
- Fault runs cover all positions only for seed 42, and always-verify only the mid position. CIs resample 17–18 games.
- Latencies are logical; only the executed restart measures live latency (27.0 s wall vs. 25.7 s model time).
- Output equality is exact, which overstates influence: the 61% figure is conservative.
- The injected faults are synthetic, the unauthorized-action policy is simulated from object types, and the fuzzing pilot is one campaign per arm.
- Two infrastructure failures were kept and rerun: one transient budget rejection from the API proxy, and four runs hit by a cache-write race in our runner, since fixed.

8 Reproducibility and cost

Everything ran on a laptop (CPU only). The binding constraint is the API rate limit (100 requests/min), not compute. All Phase 1 work used 48.6k API calls (46.3M tokens), $10.65 of TritonAI credit; the final evaluation ran for about 11 hours. The fuzzing pilot added 192 episodes and $0.81. `experiments/phase1.sh` reproduces all results and is resumable; `analysis/report.py`, `figures.py`, `diversity.py` and `restore.py` regenerate the tables and figures.